Programação Orientada a Objetos

Aula 1.1 - Revisitando lógica com Python

Prof. Dr. Eduardo de Olivindo (eduardo.olivindo@ifce.edu.br)

Tecnologia em Análise e Desenvolvimento de Sistemas

IFCE - Campus Itapipoca



Objetivos desta aula

- Revisar Python usando "novos" conteúdos.
  - Arquivos
  - Tratamento de Exceções
  - Módulos Python

Arquivos

Arquivos em Python

- O que acontece com as informações de um programa quando encerramos uma execução? → LIXO
- É necessária a **persistência** de informações em **arquivos**
- Em Python nativo há suporte a arquivos:
  - Texto puro (`.txt`, `.csv`, ...)
  - Arquivos binários (bytes)
- Todo acesso a arquivo é feito pela tríade: **Abrir** → **Manipular** → **Fechar**

Lendo dados de um arquivo `.txt`

- Use `open("caminho.txt")` para abrir arquivo de texto.
- Para fechar automaticamente o arquivo use um bloco `with...as`.
- O método `read()` lê todo o conteúdo do arquivo.

```
# Lendo tudo de uma vez
with open("dados.txt") as arq:
    conteudo = arq.read()
    print(conteudo)

# Lendo linha a linha (boa opção para arquivos grandes)
with open("dados.txt") as arq:
    for linha in arq:
        print(linha.strip()) #strip tira espaços,\n... do início e fim
```

Lendo dados de um arquivo `.txt`

Exemplo: Imprimindo linhas com numeração e contando caracteres

```
with open("dados.txt",encoding="utf-8") as new_file:
    count = 0
    total_length = 0

    for line in new_file:
        count += 1
        line = line.strip()
        print(f'linha {count}: {line}')
        total_length += len(line)

print("Quantidade total de caracteres:", total_length)
```

Lendo dados de um arquivo `.txt`

Exercício: Um arquivo chamado `numeros.txt` contém números inteiros, um por linha. O conteúdo é algo assim:

```
2
45
108
-10
1100
```

Escreva uma funcão chamada `maior` que lê este arquivo e retorna qual o maior número no arquivo.

É sempre melhor colocar o arquivo na mesma pasta que seu .py.

Lendo dados de um arquivo `.csv`

- `.csv` é texto tabular (colunas separadas por `,` , `;` . . .).
- Guarda registros de diferentes tipos (Excel, Libreoffice, Planilhas...)
- Leitura similar aos arquivos `.txt` + método `split()`
- `split("separador")` retorna uma lista de strings separadas por `separador`

```
profs = "Eduardo,Larissa,Luciano,Sérgio,Victor"
profs = profs.split(",")
for prof in profs:
    print(prof)
```

Lendo dados de um arquivo `.csv`

Exemplo: `frutas.csv` contem em cada linha nome e preço separados por `;`.
Crie um `dict` a partir dos dados do arquivo.

```
with open("frutas.csv",encoding="utf-8") as new_file:
    frutas={}
    for line in new_file:
        line = line.strip()
        line=line.split(";")
        frutas.update({line[0]: float(line[1])})
    print(frutas)
```

Lendo dados de um arquivo `.csv`

Exercício: `matrix.csv` contem uma matriz de inteiros da forma a seguir:

```
1,0,2,8,2,1,3,2,5,2,2,2
9,2,4,5,2,4,2,4,1,10,4,2
...etc...
```

Escreva duas funções: `soma_dos_elementos` e `maior_elemento`.

Escrevendo dados em um arquivo `.txt`

- Ao abrir `open` indicar um dos modos de escrita:
  - `"w"` : sobrescreve o arquivo (cria novo se não existir)
  - `"a"` : adiciona ao final sem apagar o conteúdo
- Método `write()` pra escrever no arquivo (não adiciona quebra de linha)

```
# Sobrescreve o arquivo
with open("relatorio.txt", "w", encoding="utf-8") as arq:
    arq.write("Relatorio da turma\n")
    arq.write("Media geral: 7.4\n")

# Acrescenta uma nova linha
with open("relatorio.txt", "a", encoding="utf-8") as arq:
    arq.write("Situacao: turma aprovada\n")
```

Escrevendo dados em um arquivo `.csv`

- Não tem mistério: é só separar os dados com `,` ou `;` ou ...

Escrevendo dados em arquivo

Exercício: O arquivo `matematica.csv` contem soluções para problemas.

```
Zé;2+5;7
Yovania;3-2;1
Markin;9+3;11
Cleitin;8-3;4
Mariscleia;5+5;10
...etc...
```

A estrutura é `nome;problema;resposta`. Crie a função `filtrar_solucoes()` que:

- Lê o arquivo `matematica.csv`
- Escreve as linhas corretas no arquivo `corretas.csv`
- Escreve as linhas erradas no arquivo `erradas.csv`

Tratamento de Exceções

Warnings, Erros e Exceções

E quando as coisas dão errado? Visshhh

- Nem todo problema em um programa é igual.
  - `Warning` : É um alerta; o programa pode continuar, mas exige atenção
  - `Erro` : É um problema. Impede a execução correta (ex.: sintaxe)
  - `Exceção` : Erro em tempo de execução que pode (deve) ser tratado.
- Nesta aula, o foco é em **exceções**: capturar, tratar e seguir a execução.

Warnings, Erros e Exceções

- Warning

```
>>> import regex

Warning (from warnings module):
  File "__main__", line 1
DeprecationWarning: the regex module is deprecated; please use the re module
```

- Erro

```
while True print('Hello world')
  File "<stdin>", line 1
    while True print('Hello world')
               ^^^^^
SyntaxError: invalid syntax
```

Warnings, Erros e Exceções

- Exceções

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

Validação de entrada

Muita coisa pode dar errado ao interagir com o usuário durante a execução:

- Valores de entrada ausentes ou vazios em campos obrigatórios
- Valores negativos onde apenas valores positivos são aceitos
- Arquivos ausentes ou erros de digitação em nomes de arquivos
- Valores muito pequenos ou muito grandes, por exemplo, ao trabalhar com datas e horas
- Índices inválidos, como tentar acessar o índice 3 na string "oi"
- Valores de tipo incorreto, como strings quando se esperam inteiros

Nós programadores temos que nos preparar para isso! Ahh o usuário 🥰🥰

Validação de Entrada - Exemplo

- Queremos coletar a idade de um usuário real.
- Regra: idade deve estar entre `0` e `150`.
- **RESOLVIDO!!!**

```
idade = int(input("Digite sua idade: "))
if idade>=0  and idade <= 150:
  print("Idade válida")
else:
  print("Idade fora da faixa")
```

- Maravilha!!! Tudo certo, nós somos ótimos programadores!!!!!

Validação de entrada - Exemplo

- O primeiro usuário do sistema executa assim o código::

```
Digite sua idade: vinte e tres
```

- Resultado: Quebrou e a execução encerrou

```
ValueError: invalid literal for int() with base 10: 'vinte e tres'
```

- O programa para na hora.
- Precisamos tratar essa situação para não encerrar a execução.

Tratamento com `try` / `except`

- Se algo no bloco `try` da errado o programa não para, mas executa o que está no bloco `except` correspondente

```
try:
  idade = int(input("Digite sua idade: "))
except ValueError:
  idade = -1

if idade>=0 and idade <= 150:
  print("Idade válida")
else:
  print("Idade inválida")
```

- Se a conversão falhar, cai no `except` .
- O programa continua com fluxo conhecido.

Tratamento com `try` / `except`

Exercício Crie uma função chama `ler_entrada()`, que pede que o usuário digite um valor entre dois limites passados como argumento. A função deve retornar o inteiro final válido digitado pelo usuário.

Um exemplo do uso da função é esse:

```
numero = ler_entrada("Digite um número ", 5, 10)
print("você digitou:", number)
```

```
Please type in a number: seven
You must type in an integer between 5 and 10
Please type in a number: -3
You must type in an integer between 5 and 10
Please type in a number: 8
You typed in: 8
```

Tratamento com `try` / `except`

Exercício: Resposta

```
def ler_entrada(msg, minimo, maximo):
  while True:
    try:
      valor = int(input(msg))
      if minimo <= valor <= maximo:
        return valor
    except ValueError:
      pass
    print(f"Você deve digitar um inteiro entre {minimo} e {maximo}")

numero = ler_entrada("Digite um numero: ", 5, 10)
print("Você digitou:", numero)
```

Exceções comuns em Python

- `ValueError`: valor inválido para a operação
  - Ex.: `int("abc")`
- `TypeError`: tipo de dado errado
  - Ex.: `len(10)`
- `IndexError`: índice fora do intervalo
  - Ex.: `"abc"[5]`
- `ZeroDivisionError`: divisão por zero
  - Ex.: `10 / 0`
- `FileNotFoundError`: arquivo inexistente

Tratando exceções de arquivo

- A leitura de arquivos podem gerar mais de um tipo de exceção.

```
try:
  with open("dados.txt", encoding="utf-8") as arq:
    for linha in arq:
      print(linha.strip())
except FileNotFoundError:
  print("Arquivo não encontrado")
except PermissionError:
  print("Sem permissão para abrir o arquivo")
```

- Mais de um `except` permite respostas específicas.

Lançando exceções com `raise`

- Além de capturar exceções, podemos **lançar** exceções quando os dados são inválidos.
- Isso é útil em funções: quem chama a função decide como tratar.

```
def calcular_fatorial(n):
  if n < 0:
    raise ValueError(f"Entrada inválida: {n}. Use n >= 0")

  resultado = 1
  for i in range(2, n + 1):
    resultado *= i
  return resultado

print(calcular_fatorial(5))
print(calcular_fatorial(-1))
```

Lançando exceções com `raise`

Exercício: Crie a função `nova_pessoa(nome, idade)` que retorna a tupla `(nome, idade)`.

Use `raise ValueError(...)` quando:

- `nome` for vazio
- `nome` tiver menos de duas palavras
- `nome` tiver mais de 40 caracteres
- `idade` for menor que `0` ou maior que `150`

Depois, teste com `try` / `except` para mostrar mensagens amigáveis.

Módulos Python

Módulos em Python

- Até aqui usamos funções nativas (`print` , `len` , `sum` , `open` ...)
- Em programas reais, isso não é suficiente para tudo
- Python resolve isso com **módulos**:
    - Arquivos com funções, classes e constantes prontas
    - Organizados por tema (matemática: `math` , datas: `datetime` , aleatoriedade: `random` , arquivos: `csv` , `json` , web: `urllib` ...)
- A biblioteca padrão do Python já traz vários módulos úteis
- Para usar um módulo, precisamos fazer `import`

Importando um módulo (`math`)

```
import math

print(math.sqrt(25))      # raiz quadrada
print(math.log(8, 2))     # log de 8 na base 2
print(math.pi)            # constante pi
```

- Ao importar com `import math`, usamos o prefixo `math.`
- Isso deixa claro de onde veio cada função

Importando partes específicas (`math`)

```
from math import sqrt, log

print(sqrt(49))
print(log(32, 2))
```

- Agora não precisamos do prefixo `math.`
- Útil quando vamos usar poucas funções

```
from math import *
```

- Existe, mas em geral evite em projetos maiores (pode causar conflitos de nomes)

Inspecionando conteúdo de um módulo (`math`)

- A função `dir()` mostra nomes disponíveis dentro de um módulo

```
import math

print(dir(math))
```

- Você verá constantes (`pi`, `e`) e funções (`sqrt`, `factorial`, `sin` ...)
- O caminho oficial para detalhes sempre é a documentação:
  - https://docs.python.org/3/library/

Exercício: hipotenusa

Crie uma função chamada `hipotenusa(cateto1: float, cateto2: float)` que retorna o valor da hipotenusa de um triângulo retângulo.

Regras:

- Use o Teorema de Pitágoras
- Retorne o resultado como `float`

Exemplo:

```
print(hipotenusa(3, 4))      # 5.0
print(hipotenusa(5, 12))     # 13.0
print(hipotenusa(1, 1))      # 1.4142135623730951
```

Módulo `random`

- Serve para gerar valores pseudoaleatórios
- Funções muito comuns:
  - `randint(a, b)`: inteiro entre `a` e `b` (inclusive)
  - `choice(lista)`: escolhe um item aleatório
  - `shuffle(lista)`: embaralha a lista no próprio lugar
  - `sample(populacao, k)`: escolhe `k` itens sem repetição

```
from random import randint

print("Dado:", randint(1, 6))
```

Semente aleatória (`seed`)

- Os números do módulo `random` são pseudoaleatórios
- Para testes, podemos fixar uma semente:

```
from random import randint, seed

seed(1337)
print(randint(1, 100))
print(randint(1, 100))
```

- Com a mesma semente, a sequência gerada se repete
- Ótimo para reproduzir testes

Exercício: loteria

Crie a função `numeros_loteria(qtd, menor, maior)` que:

- Sorteia `qtd` números entre `menor` e `maior`
- Não repete número
- Retorna lista ordenada

Exemplo:

```
print(numeros_loteria(6, 1, 60)) #[1,4,51,13,7,45]
```

Módulo `datetime`

- Usado para trabalhar com datas e horários
- Criação de objetos de data e hora:

```
from datetime import datetime

agora = datetime.now()
print(agora)

natal = datetime(2026, 12, 25)
print(natal)
print(natal.day, natal.month, natal.year)
```

Comparando datas e calculando diferenças

```
from datetime import datetime

hoje = datetime.now()
prova = datetime(2026, 11, 20)

if hoje < prova:
    print("A prova ainda vai acontecer")

diferenca = prova - hoje
print("Faltam", diferenca.days, "dias")
```

- A subtração de datas gera um `timedelta`
- `timedelta` traz, por exemplo, `days`, `seconds` e `microseconds`

Formatando datas (`strftime`)

```
from datetime import datetime

agora = datetime.now()
print(agora.strftime("%d.%m.%Y"))
print(agora.strftime("%d/%m/%Y %H:%M"))
```

- Códigos úteis:
  - `%d` dia
  - `%m` mês
  - `%Y` ano com 4 dígitos
  - `%H` hora (24h)
  - `%M` minuto

Convertendo texto em data (`strptime`)

```
from datetime import datetime

texto = "05.11.1986"
data = datetime.strptime(texto, "%d.%m.%Y")
print(data)
```

- `strptime` faz o caminho inverso do `strftime`
- Muito útil para entrada de usuário e leitura de arquivos

Exercício: idade em dias

Faça um programa que:

- Lê dia, mês e ano de nascimento
- Calcula quantos dias a pessoa tinha em `31/12/1999`
- Se nasceu depois dessa data, informe que ainda não havia nascido

Dica: use `datetime(...)` e subtração entre datas.

Processamento de dados com `csv`

- Para arquivos CSV, prefira o módulo `csv` em vez de `split`
- Ele lida melhor com casos como separadores dentro de strings

```
import csv

with open("frutas.csv", encoding="utf-8") as arq:
    for linha in csv.reader(arq, delimiter=";"):
        print(linha)
```

Processamento de dados com `json`

- JSON é muito usado para troca de dados entre sistemas
- Em Python, usamos o módulo `json`

```
import json

with open("cursos.json", encoding="utf-8") as arq:
    dados = json.loads(arq.read())

for curso in dados:
    print(curso["nome"])
```

- A estrutura do JSON é igual a uma lista que contêm dicionarios

Buscando dados da internet `urllib`

- Podemos obter conteúdo online com `urllib.request.urlopen`
- Muito comum quando APIs retornam JSON

```
import urllib.request, json

url = "https://randomuser.me/api/" # API que retorna dados fakes

# Faz a requisição HTTP
with urllib.request.urlopen(url) as resposta:

    dados = resposta.read().decode("utf-8") # Lê e converte para string
    infos = json.loads(dados) # Transforma a string em um dicionário
    print(infos)
```

Buscando dados da internet `urllib`

Exercício: Consulta CEP

- `https://viacep.com.br/ws/{cep}/json/`
- Acesse e mostre as informações do seu cep

Buscando dados da internet `urllib`

Consulta CEP

```
import urllib.request, json

cep = "62505090"
url = f"https://viacep.com.br/ws/{cep}/json/"

# Faz a requisição HTTP
with urllib.request.urlopen(url) as resposta:

    dados = resposta.read().decode("utf-8") # Lê e converte para string
    endereco = json.loads(dados) # Transforma a string em um dicionário
    print(endereco)
```

Exercício: Dog API 

- A Dog-API retorna links de imagens de cachorrinhos.
- Pra gerar um link dog aleatório: `https://dog.ceo/api/breeds/image/random`
- Tente mostrar a foto de um cachorrinho aleatório:
- Para mostrar a imagem use as biliotecas:

```
from PIL import Image
from io import BytesIO
```

- Após ler os dados da imagem vindos da API, mostre a foto assim:

```
img = Image.open(BytesIO(dados_da_imagem))
img.show()
```

Exercício: `pokeapi.co`



- A *Pokeapi* retorna informações de Pokémons.
- O acesso pode ser feito por:

  `https://pokeapi.co/api/v2/pokemon/{nome_do_pokemon}`
- Pesquise as informações da *pokeapi* e mostre a altura `height`, peso `weight` e a imagem de um Pikachu:
- OBS: algumas APIs podem ser chatas de entrar, nesta antes de abrir-la você deve indicar o agente da requisição http (olha redes aí...):

```
requisicao = urllib.request.Request(url_escolhida, headers={"User-Agent": "Mozilla/5.0"})
```

Onde buscar módulos

- Biblioteca padrão do Python:
  - https://docs.python.org/3/library/
- Lista de módulos úteis da comunidade:
  - https://wiki.python.org/moin/UsefulModules

Antes de usar um módulo novo, sempre confira:

- o que ele resolve
- exemplos de uso
- limitações
- versão compatível com seu Python